CS-UH 2012 - Software Engineering
Spring 2025
Final Exam
Time Limit: 120 minutes

Name: ____________________

Net ID: ____________________

This exam contains 15 pages (including this page) and 27 questions.
The total of points is 25.5. **Version A**

Part 1: Multiple Choice Questions (10.5 points)

Each question in this part is for 0.5 point.

1. Which scenario is MOST suitable for the Scrum framework over Waterfall?
   - ◯ Fixed scope and budget, flexible timelines.
   - ◯ Clear requirements, low risk of changes.
   - √ **High uncertainty and evolving needs.**
   - ◯ Long-term cycle, minimal involvement.

2. A requirements engineer bases chess game requirements on personal understanding. This is an example of:
   - √ **Introspection.**
   - ◯ Prototyping.
   - ◯ Domain Analysis.
   - ◯ Goal-based approaches.

3. Which practice ensures requirements are consistent and unambiguous?
   - ◯ Using speculative terms.
   - ◯ Including hedging phrases.
   - √ **Avoiding conjunction terms.**
   - ◯ Combining operational and descriptive specs.

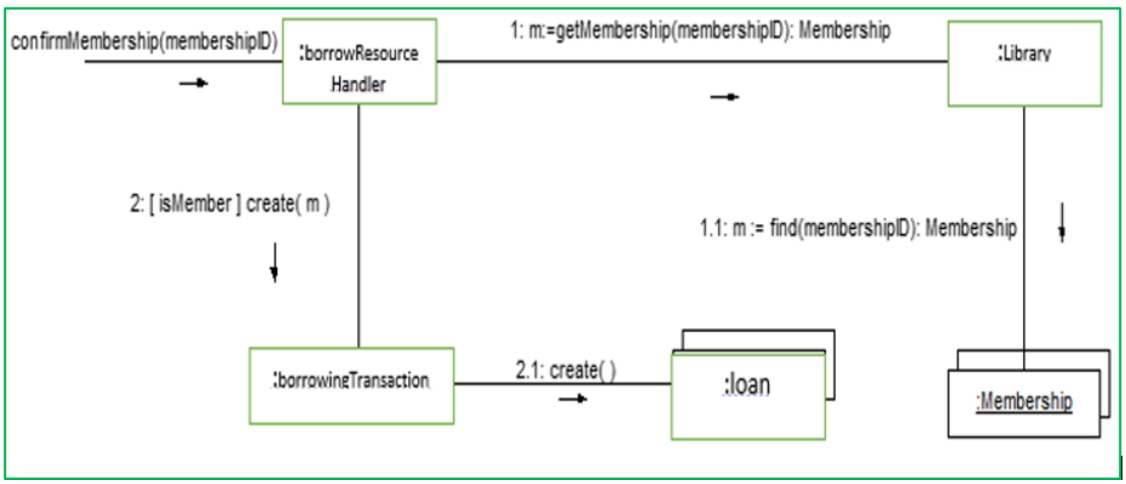
4. In a library system, a Library contains Books, and the books can exist independently of the library. What kind of association best represents this relationship?
   - √ **Aggregation.**
   - ◯ Composition.
   - ◯ Generalization.
   - ◯ Inheritance.

5. A library management system requires generating overdue notices in different formats (e.g., email, printed letter, SMS). The logic for generating each format is currently embedded in the Librarian class, making it large and difficult to maintain.

   To improve the design and better follow GRASP principles, which pattern suggests introducing a new class (e.g., NoticeFormatter) dedicated solely to handling the formatting responsibility?

   ◯ Information Expert.
   √ **Pure Fabrication.**
   ◯ Don't Talk to Strangers.
   ◯ Creator.

6. In an interaction diagram for `confirmMembership()`, which class implements the membership finding method find() used in message 1.1?



   ◯ borrowResourceHandler.
   ◯ Library.
   ◯ Membership.
   √ **None of the above.**

7. Which of the following is a potential consequence of a high spoilage metric in software testing?

   ◯ Improved code stability.
   √ **Decreased effectiveness in identifying defects.**
   ◯ Increased effectiveness in identifying defects.
   ◯ Increased customer satisfaction.

8. What is the cyclomatic complexity of the following code segment from Microsoft's Zune MP3 player software?

```
year = 1980;
while (days > 365)
{
    if (IsLeapYear(year))
    {
        if (days > 366)
        {
            days -= 366;
            year += 1;
        }
    }
    else
    {
        days -= 365;
        year += 1;
    }
}
```

√ **4.**

◯ 5.

◯ 6.

◯ 7.

9. What data flow anomaly can be detected in the following code:

```
int m = 0;
do {
    double total = 0;
    total = total + marks[m];
    m=m + 1;
} while(m < numOfMarks);
return total;
```

◯ dd (defined and then defined again).

√ **ur (undefined but referenced).**

√ **du (defined but not referenced).**

◯ None of the above,

10. What is the primary purpose of executing regression tests?

◯ Validate environment.

√ **Ensure new changes do not break existing functionality.**

◯ Test new features.

◯ Monitor defect resolution efficiency.

11. Which architectural tactic is specifically associated with enhancing the performance of a system?

    ○ Use an intermediary.

    ○ Revoke access.

    √ **Manage the sampling rate.**

    ○ Split modules.

12. Which tactic prevents brute-force attacks on authentication?

    ○ Encrypt data.

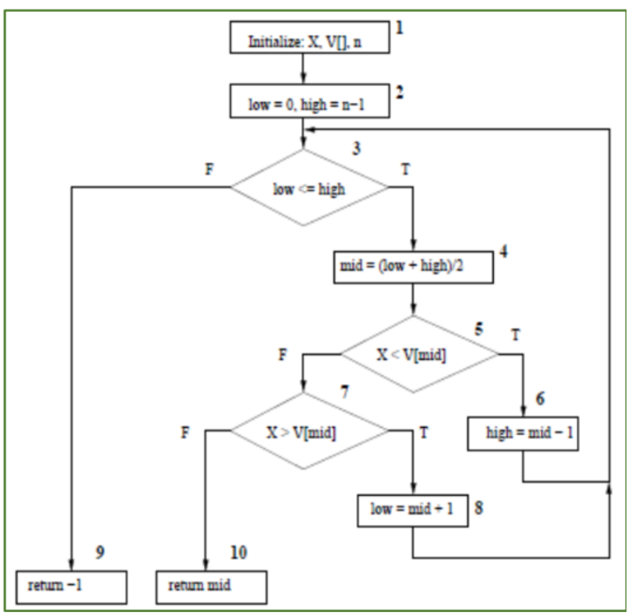
    √ **Lock accounts after failed attempts.**

    ○ Maintain audit trail.

    ○ Use two-factor authentication.

13. Which set of paths through a binary search control flow graph represents complete statement coverage?

    Path 1: 1 – 2 – 3 – 4 – 5 – 7 – 10.
    Path 2: 1 – 2 – 3 – 4 – 5 – 7 – 8 – 3 – 9.
    Path 3: 1 - 2 - 3 – 9.
    Path 4: 1 – 2 – 3 – 4 – 5 – 6 – 3 – 9.



    ○ Paths 1 and 4.

    √ **Paths 1, 2, and 4.**

    ○ Paths 1, 2, and 3.

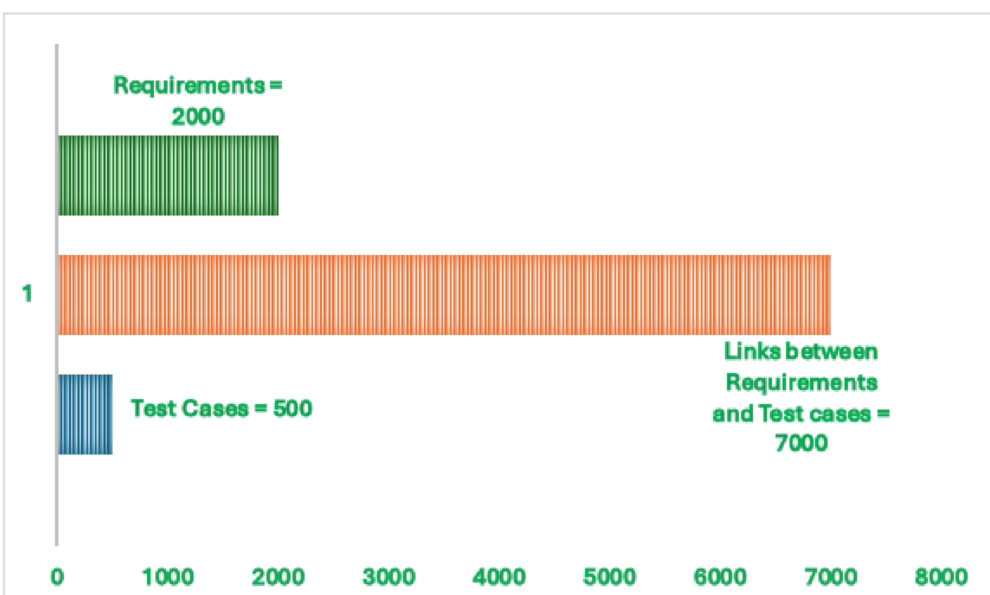
    ○ Paths 2, 3, and 4.

14. What is the minimum number of test cases for 100% branch coverage on the same graph from the previous question?
    - ◯ 1.
    - √ **2.**
    - ◯ 3.
    - ◯ 4.

15. In a Model-View-Controller (MVC) pattern, which of the following statements is true?
    - ◯ The Model handles user input and updates the View.
    - √ **The Controller updates the Model and the View based on user actions.**
    - ◯ The View directly interacts with the Model to change data.
    - ◯ The Model updates the Controller whenever the data changes.

16. You are testing the *installation* of a game on a mobile phone.
    One of the system test cases suggests testing the scenario of interrupting the installation while the battery is about to run out and so the mobile is switched off, then resuming after recharging and switching it back on.
    How do you categorize this test case?
    - ◯ Regression Testing.
    - ◯ Stress Testing.
    - √ **Robustness Testing.**
    - ◯ Load and Stability Testing.

17. Refer to the traceability matrix shown in the figure below, which illustrates the number of requirements, test cases, and links. Which of the following statements is true?



- ◯ Since the number of links is greater than requirements, each requirement is tested by at least one test case.
- ◯ Test cases are cohesive; each tests a single requirement.
- √ **The ratio between requirements and test cases is almost 4:1.**
- ◯ More links are needed to cover remaining requirements.

18. The Pipes and Filters architectural pattern is particularly beneficial for:
    - ◯ Real-time processing of large datasets.
    - √ **Sequential execution of tasks through independent stages.**
    - ◯ Hierarchical organization of components.
    - ◯ Managing control through a central coordinating component.

19. Consider a traceability matrix in the following format:

Requirement Number	**Stakeholder Requesting/Responsible**	**Prevailing Standard or Law (if any)**	**Estimated Cost**	**Priority (high, medium, low)**

and consider the following problems:

I. Identifying requirements that might need to change in the event of a change to an applicable standard.
II. Assisting in tradeoff analysis if the system cost exceeds the budget.
III. Identifying how a change in one requirement might affect another requirement.
IV. Assisting in deciding which requirement to change in order to meet a timing constraint.

Which of the problem sets could be resolved using this matrix?

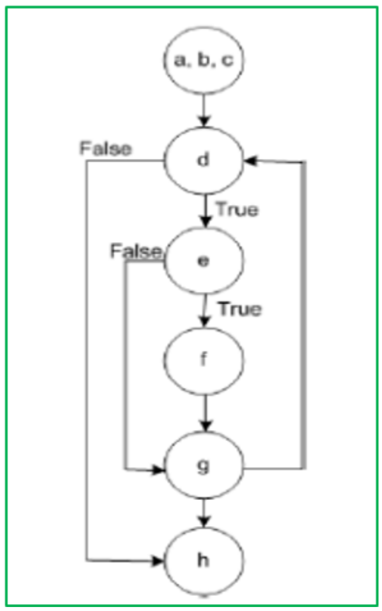
- ◯ I and III only.
- ◯ II and III only.
- √ **I and II only.**
- ◯ I and IV only.

20. In the context of redundancy as an availability tactic, what distinguishes Passive Redundancy (warm spare) from Active Redundancy (hot spare)?
    - √ **Only active members process input traffic in Passive Redundancy.**
    - ◯ Spares remain out of service until fail-over in Active Redundancy.
    - ◯ Spares maintain synchronous state at all times with active nodes in Passive Redundancy.
    - ◯ All nodes process identical inputs in parallel in Passive Redundancy.

21. How many test cases are needed for all-pairs testing of three binary parameters?
    - ◯ 3.
    - √ **4.**
    - ◯ 8.
    - ◯ 12.

Part II: Essay / Short Answers / Design Questions (15 points)

22. (1 point) For the control flow graph below:



(a) What is its associated cyclomatic complexity?

(b) What is the minimum number of test cases to achieve 100% all statement coverage?

Solution: a) 4 b) 1

23. (2 points) A smart hospital infusion pump is a programmable medical device that delivers fluids—such as medications, nutrients, or blood products—into a patient's bloodstream at precisely controlled rates and volumes. It incorporates onboard safety features like Dose Error Reduction Systems (DERS), barcode medication verification, pressure and flow-rate sensors, and built-in alarms, and can communicate with hospital information systems to automatically log dosing data and check for potential errors before and during infusion.

    – Write a partially-dressed use case for the "Administer Drug Dose" scenario, including **the main success**, and **two relevant extensions**. Ensure that the use case follows the use case guidelines covered in the class (e.g., Complete Single Goal, Leveled Steps, etc.)

24. (3 points) WeatherHub is a system that provides weather data to various apps (mobile, web). Instead of hard-coding calls to a single weather API, every client request goes through a **Broker**:

- Clients call getForecast(location) on the Broker's interface.
- The Broker maintains a registry of multiple **WeatherService** providers (e.g., ServiceA, ServiceB, ServiceC) and their current status.
- Upon each request, the Broker selects an appropriate WeatherService (based on availability, response time, or quota) and forwards the request, then returns the provider's response to the client.

In the context of the WeatherHub scenario above, consider the use of the **Broker Pattern** for client–server communication:

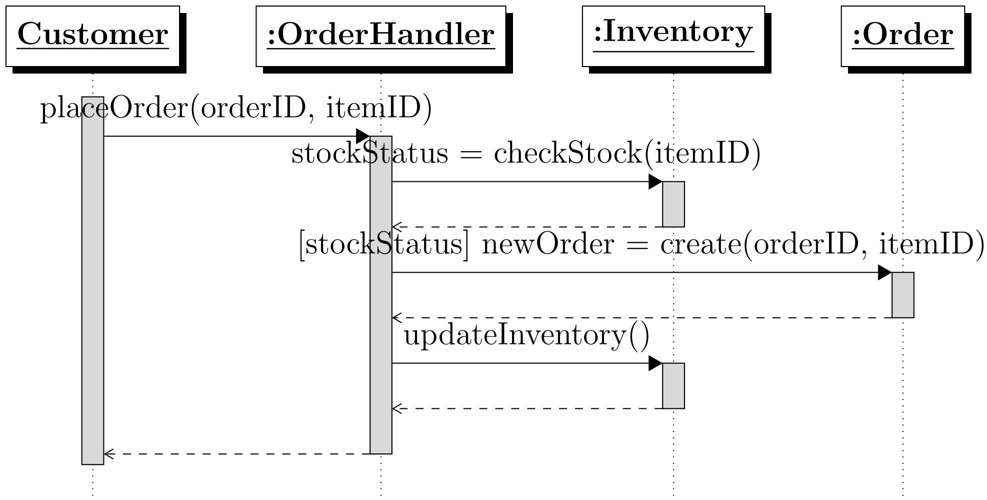
(a) Identify one specific **strength** of placing the Broker between clients and multiple WeatherService providers. Justify your answer with reference to the scenario (1 point).

(b) Identify one specific **weakness** introduced by the Broker in WeatherHub's architecture. Explain why this is a concern in the scenario (1 point).

(c) Select one architectural **tactic** that directly addresses the identified weakness. Describe how applying this tactic in WeatherHub would mitigate the problem (1 point).

25. (3 points) Consider the following sequence diagram for the `placeOrder(orderID, itemID)` method. Please note that *Customer* represents an external primary actor.

Implement the `placeOrder` method based on the diagram. Use pseudocode, C++, or Java (focus on logic).



Solution: One possible solution, in Java, could be:

```
public void placeOrder(String orderID, String itemID) {
```

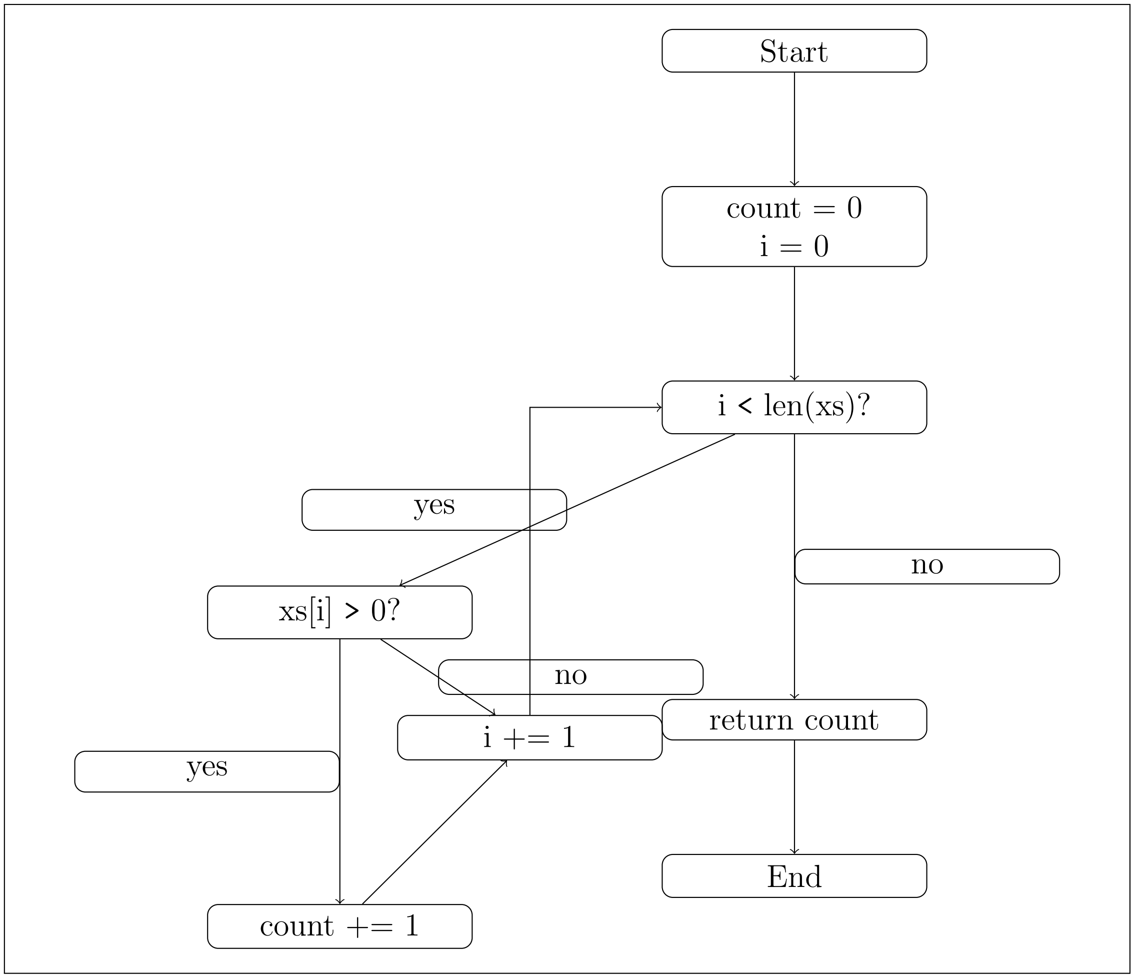
```
    boolean stockStatus = inventory.checkStock(itemID);
    if (stockStatus) {
        Order newOrder = order.create(orderID, itemID);
        inventory.updateInventory();
    }
}
```

26. (4 points) Consider the following Java function:

```java
public static int countPositives(int[] xs) {
    int count = 0;
    int i = 0;
    while (i < xs.length) {
        if (xs[i] > 0) {
            count += 1;
        }
        i += 1;
    }
    return count;
}
```

(a) Draw a control flow graph (CFG) for this function. You can group statements as long as your answer shows clearly the groupings that you made. (2 points)

Solution: Below is a control flow graph (CFG) for the function `count_positives(xs)`. We group related statements into basic blocks.



(b) Provide a concrete test suite for the program above that achieves 100% branch (edge) coverage. The test suite should have the minimal number of test cases that are necessary to achieve the coverage criteria. Explain your answer. (2 points)

Solution: A minimal test suite is:

- `[1, -2]` or any list containing both positive and non-positive values

27. (2 points) Consider ShopEase, a cloud-hosted e-commerce platform where customers can browse products, place orders, pay via a third-party gateway, and request refunds through a REST API.

    (a) Define the concept of **repudiation** in security testing and give an example in the context of ShopEase platform.

    (b) At the architectural level, describe one mitigation technique ShopEase could adopt to reduce repudiation risk. Explain your answer.